Exploring CMS Hospital Data Through an API

Hospital information, mortality, readmissions, and historical payments

Project exploration

2026-11-11

1 Overall plan

This document adapts the structure of `attempting_stroke_api.qmd`: define an API function, retrieve CMS datasets, and inspect the results. It adds state filtering, pagination, CSV exports, and historical payments. It is an exploratory data-access demonstration for a patient-facing Shiny hospital comparison app, not a hospital ranking.

We retrieve four datasets from the **CMS Provider Data Catalog**:

1. **Hospital General Information**: hospital identity, location, type, and overall quality rating.
2. **Complications and Deaths - Hospital**: condition-specific mortality and complication measures.
3. **Unplanned Hospital Visits - Hospital**: readmissions, return days, and other unplanned-visit measures.
4. **Historical Payment and Value of Care - Hospital**: Medicare episode payments and comparison categories.

The sample defaults to Maryland. These are hospital-level aggregates, not individual patient records. The mortality and unplanned-visit tables include multiple conditions; they are not restricted to stroke.

Payment limitation: CMS removed the disease-specific payment measures from reporting starting in July 2025. The endpoint still returns historical records. Their dates must be retained, and the values must not be described as current prices or patient out-of-pocket costs.

2 Set up R

Install the required packages once, outside the rendering process:

```
install.packages(c("httr2", "dplyr", "purrr", "readr", "tibble", "knitr"))
```

```
library(httr2)
library(dplyr)
library(purrr)
library(readr)
library(tibble)
library(knitr)

state <- toupper(params$state)
stopifnot(grepl("^[A-Z]{2}$", state))
output_dir <- paste0("cms_extract_", state)
dir.create(output_dir, recursive = TRUE, showWarnings = FALSE)
```

The API needs **no API key**: these public records can be requested without a secret access credential. We use `httr2` to send HTTP requests and parse the JSON response. The records are in `results`; `count` reports the total matching rows.

3 Create a reusable API function

A single request retrieves only one page. This function requests up to 500 records at a time, increases `offset`, and stops after retrieving the full state-filtered result. It retries transient request failures and checks completeness.

```
get_cms_data <- function(dataset_id, state, page_size = 500L) {
  url <- paste0(
    "https://data.cms.gov/provider-data/api/1/",
    "datastore/query/", dataset_id, "/0"
  )
  offset <- 0L
  pages <- list()

  repeat {
    response <- request(url) |>
      req_url_query(
        limit = page_size,
        offset = offset,
        `conditions[0][property]` = "state",
        `conditions[0][value]` = state,
        `conditions[0][operator]` = "="
      ) |>
      req_timeout(90) |>
      req_retry(max_tries = 4) |>
      req_perform()

    result <- resp_body_json(response, simplifyVector = FALSE)
    total <- as.integer(result$count)
    rows <- result$results
    if (!length(rows)) break

    pages[[length(pages) + 1L]] <- bind_rows(rows)
    offset <- offset + length(rows)
    if (offset >= total) break
  }

  data <- bind_rows(pages)
  if (!nrow(data)) stop("No records returned for ", dataset_id)
  stopifnot(nrow(data) == total, all(data$state == state))
  data
}
```

4 Retrieve and export the datasets

`imap()` applies the same acquisition function to each named dataset. This is a functional programming approach: one reusable function is mapped over a collection rather than four separate copies of the request code.

```
dataset_ids <- c(
  hospitals = "xubh-q36u",
  complications_deaths = "ynj2-r877",
  unplanned_visits = "632h-zaca",
  historical_payments = "c7us-v4mf"
)

get_or_read <- function(id, name) {
  path <- file.path(output_dir, paste0(name, ".csv"))
  if (!params$refresh && file.exists(path)) {
    return(read_csv(path, col_types = cols(.default = "c")))
  }
  data <- get_cms_data(id, state)
  write_csv(data, path)
  data
}

cms_data <- imap(dataset_ids, get_or_read)
hospitals <- cms_data$hospitals
complications_deaths <- cms_data$complications_deaths
unplanned_visits <- cms_data$unplanned_visits
historical_payments <- cms_data$historical_payments

inventory <- tibble(
  dataset = names(cms_data),
  rows = map_int(cms_data, nrow),
  columns = map_int(cms_data, ncol)
)
kable(inventory, caption = "Downloaded dataset dimensions")
```

Table 1: Downloaded dataset dimensions

dataset	rows	columns
hospitals	56	38
complications_deaths	960	18
unplanned_visits	672	20
historical_payments	180	22

The following tables show `head()` (the first six rows), with selected columns to keep the PDF readable. The exported CSVs preserve **all columns**. These first rows are previews, not a random sample or the best-performing hospitals.

5 Hospital information: first six rows

One row represents one hospital. Overall stars describe the hospital as a whole, not its performance for a particular condition.

```
hospitals |>
  select(facility_id, facility_name, citytown,
         hospital_overall_rating) |>
  head() |>
  kable(col.names = c("ID", "Hospital", "City", "Stars"),
        caption = "Hospital General Information: first six rows")
```

Table 2: Hospital General Information: first six rows

ID	Hospital	City	Stars
210001	MERITUS MEDICAL CENTER	HAGERSTOWN	3
210002	UNIVERSITY OF MARYLAND MEDICAL CENTER	BALTIMORE	3
210003	UNIVERSITY OF MD CAPITAL REGION MEDICAL CENTER	UPPER MARLBORO	1
210004	HOLY CROSS HOSPITAL	SILVER SPRING	2
210005	FREDERICK HEALTH HOSPITAL	FREDERICK	3
210008	MERCY MEDICAL CENTER INC	BALTIMORE	3

6 Mortality and complications: first six rows

One row represents a hospital and a measure. The dataset includes complications as well as mortality, so read the measure ID before interpreting the score. The same hospital appears multiple times. Scores are preserved as CMS text, including unavailable values.

```
complications_deaths |>
  select(facility_id, measure_id, score, start_date, end_date) |>
  head() |>
  kable(col.names = c("ID", "Measure", "Score", "Start", "End"),
        caption = "Complications and Deaths: first six rows")
```

Table 3: Complications and Deaths: first six rows

ID	Measure	Score	Start	End
210001	COMP_HIP_KNEE	4.3	04/01/2023	03/31/2025
210001	Hybrid_HWM	3.6	07/01/2024	06/30/2025
210001	MORT_30_AMI	12.2	07/01/2022	06/30/2025
210001	MORT_30_CABG	Not Available	07/01/2022	06/30/2025
210001	MORT_30_COPD	9	07/01/2022	06/30/2025
210001	MORT_30_HF	11.5	07/01/2022	06/30/2025

6.1 Optional stroke-specific preview

The original document's title refers to stroke. Here we explicitly select `MORT_30_STK`, rather than assuming the entire dataset is about stroke. A stroke mortality score of 12 means 12%, not 0.12. This is a reported hospital-level estimate, not an individual's risk.

```
stroke_mortality <- complications_deaths |>
  filter(measure_id == "MORT_30_STK")

stroke_mortality |>
  select(facility_id, measure_id, score, denominator) |>
  head() |>
  kable(col.names = c("ID", "Measure", "Score (%)", "Denominator"),
        caption = "Stroke mortality: first six matching rows")
```

Table 4: Stroke mortality: first six matching rows

ID	Measure	Score (%)	Denominator
210001	MORT_30_STK	12.4	284
210002	MORT_30_STK	10.2	452
210003	MORT_30_STK	10.3	211
210004	MORT_30_STK	10.5	257
210005	MORT_30_STK	11.6	328
210008	MORT_30_STK	13.1	52

7 Unplanned visits: first six rows

One row represents a hospital and an unplanned-visit measure. This includes return days and other outcomes as well as readmission rates. A negative return-day score should not be interpreted as a negative percentage. Match each score to its measure definition.

```
unplanned_visits |>
  select(facility_id, measure_id, score, start_date, end_date) |>
  head() |>
  kable(col.names = c("ID", "Measure", "Score", "Start", "End"),
        caption = "Unplanned Hospital Visits: first six rows")
```

Table 5: Unplanned Hospital Visits: first six rows

ID	Measure	Score	Start	End
210001	EDAC_30_AMI	24.7	07/01/2022	06/30/2025
210001	EDAC_30_HF	15.3	07/01/2022	06/30/2025
210001	EDAC_30_PN	39.4	07/01/2022	06/30/2025
210001	Hybrid_HWR	Not Available	07/01/2024	06/30/2025
210001	OP_32	14.1	01/01/2022	12/31/2024
210001	OP_35_ADM	14.2	01/01/2024	12/31/2024

There is no corresponding condition-specific stroke readmission measure in our selected sample. Do not label a hospital-wide readmission measure as a stroke-specific outcome.

8 Historical payments: first six rows

These records contain **Medicare episode payment amounts**. They are not charges, current price quotes, or a patient's personal bill. Payment periods may differ from mortality and readmission periods. Keep this historical view separate from current comparisons.

```
historical_payments |>
  select(facility_id, payment_measure_id, payment,
         start_date, end_date) |>
  head() |>
  kable(col.names = c("ID", "Measure", "Payment", "Start", "End"),
        caption = "Historical Payment and Value of Care: first six rows")
```

Table 6: Historical Payment and Value of Care: first six rows

ID	Measure	Payment	Start	End
210001	PAYM_30_AMI	$25,710	07/01/2020	06/30/2023
210001	PAYM_30_HF	$18,168	07/01/2020	06/30/2023
210001	PAYM_30_PN	$20,314	07/01/2020	06/30/2023
210001	PAYM_90_HIP_KNEE	$24,773	07/01/2020	03/31/2023
210002	PAYM_30_AMI	$30,291	07/01/2020	06/30/2023
210002	PAYM_30_HF	$20,745	07/01/2020	06/30/2023

9 What can be used in the Shiny app?

Join hospital characteristics to a selected outcome using `facility_id`, kept as text to preserve leading zeros. Filter to the intended measure before joining; joining two full measure tables by hospital ID alone creates many-to-many combinations.

The initial app could show condition-specific mortality and available readmission measures, overall stars, national comparisons, missing values, and reporting dates. Retain footnotes and uncertainty estimates. Unavailable values are not zeros, and small differences in reported rates do not establish that one hospital is better.

For a manageable first version, consider heart failure, heart attack, and pneumonia. Historical payments can be an optional, clearly dated exploration rather than part of a current recommendation score.

10 Reproduce this document

From the project root, run:

```
quarto render output/pdf/cms_api_extraction.qmd --to pdf
```

By default, rendering makes live API requests and writes four CSVs in `cms_extract_MD/` beside this document. After a successful download, render from those saved CSVs without requesting them again:

```
quarto render output/pdf/cms_api_extraction.qmd --to pdf -P refresh:false
```

To retrieve another state, use `-P state:PA`. The default API workflow uses R only. The PDF requires Quarto, the listed R packages, and a LaTeX installation such as TinyTeX.

11 References

- Reference structure: the supplied `attempting_stroke_api.qmd`.
- CMS Provider Data Catalog and API information.
- Hospital General Information.
- Complications and Deaths - Hospital.
- Unplanned Hospital Visits - Hospital.
- Historical Payment and Value of Care.
- CMS hospital data dictionary and reporting changes.